Dynamic model of F450 Quadcopter in Simulink

Yong Sin Lee[1*]

Key Words : Quadcoptor(쿼드콥터), Dynamic Modeling(동적 모델링), Motor Dynamics(모터 동역학), AeroDynamics(공기역학), Transfer Function(변환행렬)

서 론

최근 무인항공기 기술은 다양한 산업 분야로 빠르게 확장되면서, 높은 비행 안정성과 정밀한 위치 제어 능력이 요구되고 있다. 그러나 쿼드콥터는 주조적으로 외란에 민감한 특성을 지닌다. 이러한 문제를 해결하기 위해서는 실제 비행 실험 이전에 제어 알고리즘을 충분히 검증하여 안정성을 확보하는 과정이 필수적이다. 특히 시뮬레이션은 하드웨어 파손이나 인명 피해 없이 다양한 비행 환경을 모사할 수 있다는 점에서 효과적이다. 따라서 본 연구에서는 MATLAB/Simulink 기반의 시뮬레이션 환경을 구축하여, 실제환경 모델의 조건을 입력하고 이에 따른 쿼드콥터의 자세(Attitude), 위치(Position) 및 속도(Velocity) 응답 특성을 분석하고자 한다.

본 론

1. Motor DynamicFigure

MotorDynamics는 쿼드콥터 추진 시스템의 핵심으로, 각 모터의 회정속도 변화가 추력과 토크에 직접적인 영향을 미친다. 본 절에서는 모터의 전기적입력과 출력 간의 관계를 수학적으로 모델링하였다.

DJI F450 기준 주요 파라미터는 아래와 같다.

Table 1. Parameter of DJI F450

Symbol	Value	Unit	Description
m	1.5	$kg]$	mass
L	0.225	$[m]$	arm length
b	1.1768×10	$[\quad s^2]$	Thrust Coefficient
d	$1.8542\times10^{-}$	$[N \quad m \quad s^2$	Drag Coefficient
J	2.8×10^{-5}	$[N \quad m \quad s^2$	(Motor & Propeller) Inertia
R	0.117	$[Ohm]$	Motor Resistance
K	960	$[RPM/V]$	Motor Velocity Constant
I	diag[0.02,0.02,0.025]	$[kg \quad m^2]$	Inertia I

1-1 배경이론

모터의 전기적 모델은 키르히호프 전압법칙을 적용하여 다음과 같이 나타낼 수 있다.

$$-V_a+V_R+V_L+e=0,\ L\frac{di}{dt}=v_a-Ri-K_e w$$

여기서 e는 역기전압이며 L은 인덕턴스이다. 전기적 특성은 물리적 특성보다 매우 빠르고 인덕턴스 값은 매우 작기 때문에 0으로 근사할 수 있다. 따라서 보터의 전류는 다음과 같이 입력전압, 역기전압, 저항의 식으로 나타난다.

$$i=\frac{v_a-K_e w}{R} \tag{1}$$

회전운동 방정식은 뉴턴 제 2법칙에 의해 다음과 같이 표현된다.

$$J\frac{dw}{dt}=\tau \quad Q,\ \tau=K_t\times i$$

여기서 모터 축에 작용하는 토크는 모터토크상수와 전류를 곱한 값이며, $K_t=1/\frac{2\Pi}{60}K_v$이다.

Q는 프로펠러에서 발생하는 항력토크이며 항력토크는 프로펠러 항력계수와 각속도제곱의 곱으로 나타낼 수 있다.

$$Q=d\times\omega^2$$

따라서 이를 종합하면 전체 모터시스템의 동역학은 다음과 같이 나타낼 수 있다.

$$J\frac{dw}{dt}=K_t\frac{v_a-K_e w}{R}-dw^2 \tag{2}$$

여기서 이번 시뮬레이션에서는 배터리 조건을 고려하지 않기 때문에 전력입력 대신 호버링모터각속도를 입력하여 시뮬레이션할 계획이다. 한 모터에서 추력은 추력계수(b)와 각속도 제곱의 곱이며 쿼드콥터에서는 4개의 프로펠러를 사용하므로 전체 추력은 $4bw^2$이고 호버링 시 이는 중력과 같다. 따라서 위 식의 각속도를 $w=\frac{mg/b}{4}$. 각가속도를 0으로 치환하면 다음과 같은 필요전압을 구할 수 있다.

$$v_a=K_e w_d+\frac{R}{K_t}dw_d^2 \tag{3}$$

또한 모터에서 입력전력은 모터의 출력파워와 손실의 합이며, 이는 각각 $P=v_a i=(K_e w+iR)i$, $P=\tau w=K_t iw$이다. 여기서 손실을 0으로 가정한다면 $K_t=K_e$이며 각가속도는 다음과 같다.

$$\frac{dw}{t} = K\frac{v_a - K_t w}{R} - dw^2 \qquad (4)$$

1-2 시뮬링크 구성

모델 초기화 함수:

```
%% enviromental parameter

g = 9.81;    %[m/s^2]

%% vehicle parameter

mass=1.5;        %[kg]
Arm_Length=0.225;   %[m]

%% Motor Parameter

%%parameter
rpm2rad = 2*pi/60;
rad2rpm = 1/rpm2rad;

Friction_Coef=0.0001; %[N m/s]
Inertia=2.8e-05;      %[kg m^2]
Resistance = 0.1117;   %[ohm]
K_Torque=1/(rpm2rad*960); %[N m/A]
K_Emf= K_Torque;    %[V s/rad]
Tau = (Inertia*Resistance) /(Resistance*Friction_Coef+K_Torque*K_Emf); %[N m]
Ks=K_Torque/(Resistance*Friction_Coef+K_Torque*K_Emf);

Thrust_Coef=1.1768e-05;
Drag_Coef=1.8542e-07;
Hovering_Motor_Speed = sqrt(mass*g/4/Thrust_Coef); %[m/s]
```

Fig. 1. Simulink itial Function

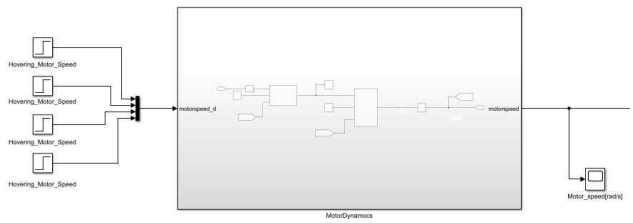


Fig. 2. Motor Dynamics Subsystem

우선 모델 초기화 함수를 통해 모델의 무게, 암 길이 등 고유값 변수와 환경 변수를 입력해주었고 이를 통해 호버링모터스피드를 계산하였다. 이때 호버링 모터스피드는 559.02로 계산되었다. 또한 모터 동역학 서브시스템을 구성하여 이에 스텝을 통해 스텝시간 1초에 호버링 모터스피드값을 입력해주었다.

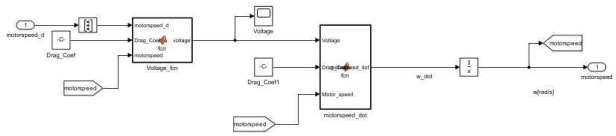


Fig. 2. Motor Dynamics Subsystem

모터동역학 서브시스템에서는 호버링모터스피드값 입력을 통해 필요전압과 모터 각가속도를 구하고 이를 적분하여 모터스피드 값을 도출해주는 시스템을 구성하였다. 이때 쿼드콥터는 모터의 개수가 4개이므로 Reshape 블록을 사용하였고 Voltage_fcn, motorspeed_dot fcn 두 개의 매트랩 Fcn블록을 사용하였으며 도출된 가속도값을 goto,from 블록을 사용해 입력하였다.

```
function voltage = fcn(motorspeed_d, Drag_Coef, motorspeed)

%%parameter
rpm2rad = 2*pi/60;
rad2rpm = 1/rpm2rad;


Resistance = 0.1117;   %[ohm]
K_Torque=1/(rpm2rad*960); %[N m/A]
K_Emf= K_Torque;    %[V s/rad]


voltage = K_Emf.*motorspeed_d+Resistance*Drag_Coef/K_Torque.*motorspeed.^2;
```

Fig. 3. Voltage_fcn

Voltage_fcn에서는 호버링모터스피드값 입력과 (3)식을 통해 필요전압을 계산하는 함수를 구성하였다.

```
function motorspeed_dot= fcn(Voltage,Drag_Coef, Motor_speed)


%%parameter
rpm2rad = 2*pi/60;
rad2rpm = 1/rpm2rad;


Inertia=2.8e-05;      %[kg m^2]
Resistance = 0.1117;   %[ohm]
K_Torque=1/(rpm2rad*960); %[N m/A]
K_Emf= K_Torque;    %[V s/rad]

V_Emf=K_Emf .* Motor_speed;
Current=(Voltage-V_Emf)/Resistance;
Motorspeed_dot=(K_Torque*Current-Drag_Coef.*Motor_speed.^2)/Inertia;

motorspeed_dot = Motorspeed_dot;
```

Fig. 4. Motorspeed_dot_fcn

motorspeed_doc_fcn에서는 (4)식을통해 모터의 각가속도를 계산하는 함수를 구성하였다.

1-3 결과

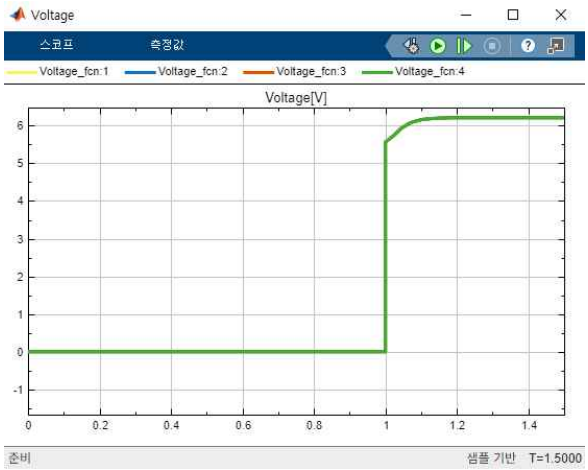


Fig. 5. Voltage_Result

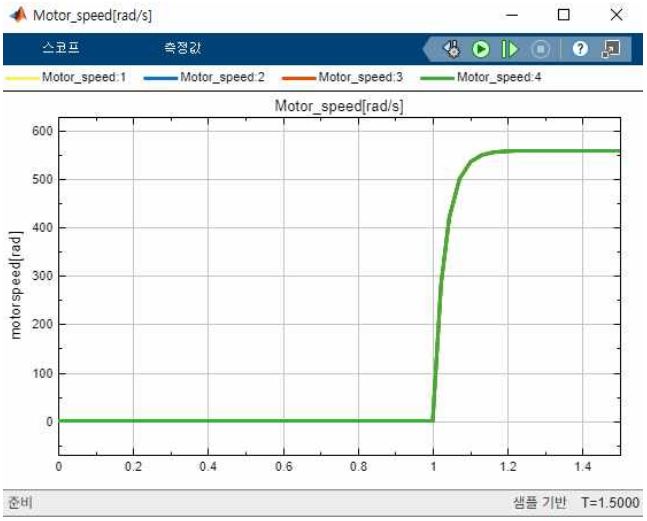


Fig. 6. Motor_speed_Result

모터시스템으로 시뮬링크를 실행한 결과 1초에 호버링각속도가 스텝으로 입력되었을 때 필요전압은 6V, 모터각속도는 550부근으로 수렴하는 것을 확인할 수 있다.

2. AeroDynamicFigure

2-1 배경이론

한 모터에서는 프로펠러의 회전으로 다음과 같은 추력이 발생한다.

$$\frac{1}{2}\rho C A(Rw)^2 = bw^2 \tag{5}$$

쿼드콥터는 4개의 프로펠러 각속도을 조절하여 바디에 작용하는 힘과 모멘트를 생성한다. 이때 추력은 바디의 수직방향으로만 존재하며 모멘트는 프로펠러회전의 반작용을 통해 생성된다. 각 프로펠러의 번호와 회전방향은 다음과 같다.

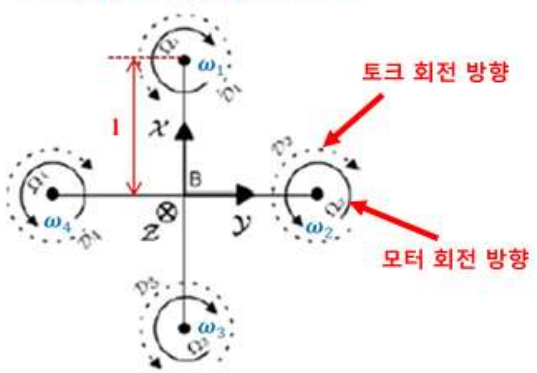


Fig. 7. Quadcopter configuration

바디 좌표계 기준으로 드론에 작용하는 총 추력 벡터는 다음과 같이 정의된다.

$$F = \begin{bmatrix} 0 \\ 0 \\ -b(w_1^2 + w_2^2 + w_3^2 + w_4^2) \end{bmatrix} \tag{6}$$

즉, 모든 프로펠러의 추력이 z-축 방향으로만 합산되어 상승력을 제공하며, 중력에 대항한다.

각 모터의 회전속도에 따라 발생하는 모멘트는 다음과 같다.

$$M_B = \begin{bmatrix} Lb(w_4^2 - w_2^2) \\ Lb(w_1^2 - w_3^2) \\ d(-w_1^2 + w_2^2 - w_3^2 + w_4^2) \end{bmatrix} \tag{7}$$

여기서 L은 로터중심에서 무게중심까지의 거리인 암길이를 의미한다. 첫 번째부터 각각 Roll,Pitch,Yaw축의 모멘트를 나타내며 Roll회전은 4번과2번 모터의 각속도 차이로 y축 회전은 1번모터와 3번모터의 각속도 차이로 발생한다.

외란이 존재하지 않을 때 드론에 작용하는 힘과 모멘트는 (6)번,(7)번 행렬과 중력의 합으로 나타낼 수 있다. 이때 바디에 작용하는 중력은 다음과 같은 변환행렬을 적용하여 로컬 좌표계로 나타낼 수 있다.

$$R_B^E = R_z \cdot R_y \cdot R_x = \begin{bmatrix} c(\psi)c(\theta) & c(\psi)s(\theta)s(\varphi) - s(\psi)c(\varphi) & c(\psi)s(\theta)c(\varphi) + s(\psi)s(\varphi) \\ s(\psi)c(\theta) & s(\psi)s(\theta)s(\varphi) + c(\psi)c(\varphi) & s(\psi)s(\theta)c(\varphi) - c(\psi)s(\varphi) \\ -s(\theta) & c(\theta)s(\varphi) & c(\theta)c(\varphi) \end{bmatrix}$$

Fig. 8. Transfer matrix

2-2 전력효율,비행시간

모멘텀이런에 따르면, 로터가 공기를 가속시키며 추력을 발생시킬 때, 추진력은 다음과 같이 정의된다.

$$T = 2\rho A v_i^2 \tag{8}$$

여기서 v_i는 유도속도이며, 공기 흐름의 운동량 변화에 의해 발생하는 소도 성분이다. 이때 이상적 소비전력은

$$P_{ideal} = Tv_i = \frac{T^{\frac{3}{2}}}{2\rho A} \tag{9}$$

로 나타낼 수 있다. 실제 프로펠러의 효율 손실을 고려하면, Figure of Merit(FM)을 적용하여 다음과 같이 보정된다.

$$P_{real} = \frac{P_{ideal}}{FM} \tag{10}$$

해당 내용은 따로 m파일로 작성하여 계산하였다.

```
R = 0.119;
FM = 0.4;
U_batt = 5;
V_batt = 14.8;
E_batt = U_batt *V_batt;
m=1.5;
g =9.8;
pfl =1.25; %kg/m^3

Thrust = 1/4 *m*g
P_ideal = Thrust^(3/2)/sqrt(2*pfl*R^2*pi)
P_real = 4*P_ideal/FM
time = E_batt/P_real;
time_min =60*time
```

```
Thrust =

    3.6750


P_ideal =

   21.1249


P_real =

  211.2487


time_min =

   21.0179
```

Fig. 9. Power & Fling Time of Quadcopter

FM = 0.4, DJI F450 기준 $_{rop}$ $\frac{1}{4}mg$일 때, 실제 소비전력은 211.2W 최대 비행시간은 21분으로 계산되었다.

2-3 시뮬링크 구성

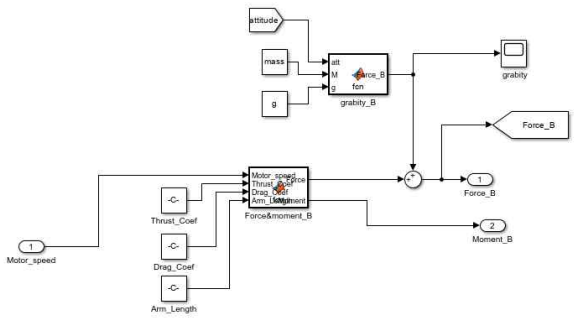


Fig. 10. Simulik of AeroDynamics Subsystem

Fig9는 AeroDynamic 서브시스템 내부 구성이다. Force&moment_B , grabity_B 매트랩fcn을 통해 바디좌표계 프로펠러 추력을 인한 힘과 모멘트, 중력을 구하고 이를 sum블록으로 합쳐 도출해내는 구성을 만들었다.

```
function [Force , Moment] = fcn(Motor_speed,Thrust_Coef,Drag_Coef,Arm_Length)

Force = [0;
    0;
    -Thrust_Coef * (Motor_speed(1).^2+Motor_speed(2).^2+Motor_speed(3).^2+Motor_speed(4).^2)];
Moment = [ Arm_Length * Thrust_Coef * (Motor_speed(4).^2 - Motor_speed(2).^2);
    Arm_Length * Thrust_Coef * (Motor_speed(3).^2 - Motor_speed(1).^2);
    Drag_Coef * (-Motor_speed(1).^2+Motor_speed(2).^2-Motor_speed(3).^2+Motor_speed(4).^2)];
```

Fig. 11. Force&moment_B fcn

```
function Force_B = fcn(att,m,g)

phi=att(1)*pi/180;
theta=att(2)*pi/180;
psi=att(3)*pi/180;

T=[0,0,m*g]';

R = [cos(theta)*cos(psi), cos(theta)*sin(psi), -sin(theta);
     sin(phi)*sin(theta)*cos(psi)-cos(phi)*sin(psi), sin(phi)*sin(theta)*sin(psi)+cos(phi)*cos(psi), sin(phi)*cos(theta);
     cos(phi)*sin(theta)*cos(psi)+sin(phi)*sin(psi), cos(phi)*sin(theta)*sin(psi)-sin(phi)*cos(psi), cos(phi)*cos(theta)];

Force_B = R*T;
```

Fig. 12. grabity_B

fig11,fig12은 각각 Force&moment_B , grabity_B 매트랩fcn 내부모습이다. 첫 번째에서는 (6)번과 (7)번식을 통해 프로펠러로 인한 바디좌표계 힘과모멘트를 도출하였으며, 두 번째 fcn에서는 fig8의 변환행렬을 통해 바디좌표계 중력을 도출해준다.

스타일 (세부절 제목), 본문내용은 (스타일 초록 및 본문내용)을 따라 작성한다.

2-4 결과

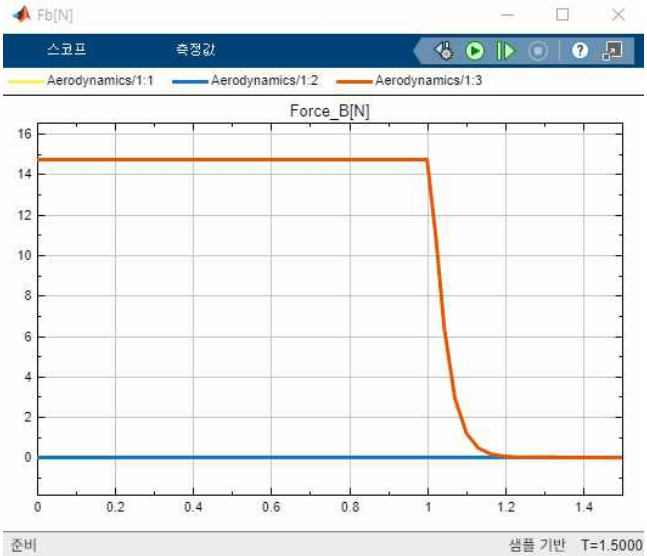


Fig. 13. Force_B [N]

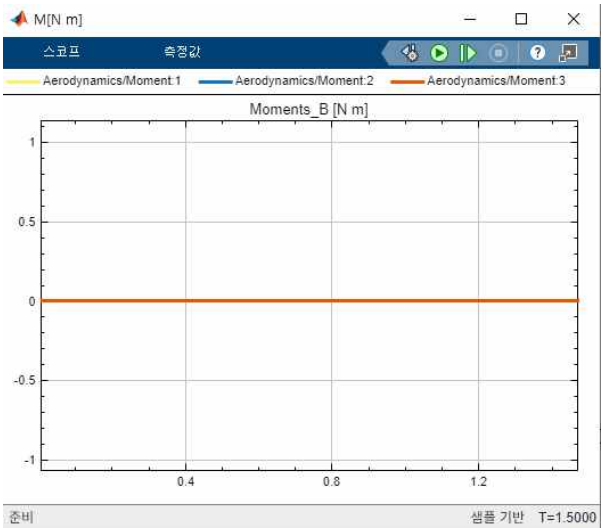


Fig. 14. Moments_B [N m]

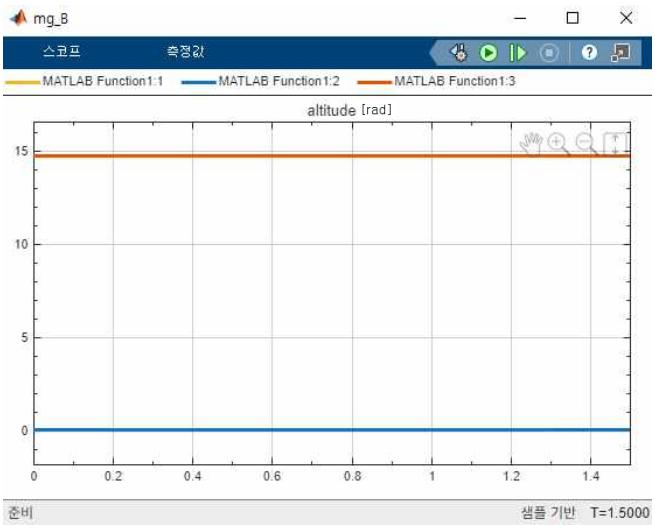


Fig. 15. grabity_B [N]

Fig 13,14,15는 각각 바디좌표계의 힘,모멘트,중력을 그래프로 나타낸 것이다. 해당 조건의 경우 호버링 상태 즉, 4개프로펠러의 각속도가 모두 동일하므로 모멘트는 0이며 자세가 평형하므로 z축으로만 중력이 작용한다. 또한 Fig13을 보았을 때 중력이 작용하던 중 1초에 모터각속도를 입력하였을 때 전체 Force_B값이 빠르게 줄어들어 0으로 수렴함을 확인할 수 있다.

Fig9는 AeroDynamic 서브시스템 내부 구성이다. Force&moment_B , grabity_B 매트랩fcn을 통해 바디좌표계 프로

여기에 후기를 작성한다. (마지막 줄은 공란)

3. BodyDynamics & Kinematics

3-1 Body Dynamics

쿼드콥터는 6자유도 운동방정식을 기반으로 한 물체 동역학을 기술한다. 쿼드콥터는 강체운동을 기반으로 하며, 물체 중심에 정의된 바디 좌표계와 지구 기준의 이너셜좌표계 사이의 변환을 통해 병진 및 회전 운동을 해석한다.

병진 운동 방정식은 뉴턴 제 2법칙에 따라 다음과 같이 나타낸다.

여기서 m은 쿼드콥터의 질량,속도벡터는 $\frac{d}{dt} = F$이다. (11)

이너셜 좌표계가 아닌 바디 좌표계에서의 시간 미분을 취하면, 다음과 같은 형태로 표현된다.

$$m\left(\frac{dV}{dt_b} + w \times V\right) = F \quad (12)$$

여기서 w는 $w = [p, q, r]$ 바디좌표계에서의 각속도를 의미한다.

우변의 힘 벡터 F는 다음과 같이 구성된다.

$$F = F_{aero} + F_{gravity} + F_{thrust} \quad (13)$$

즉, 공기역학적 항력, 중력, 프로펠러에 의해 생성되는 추력의 합으로 표현된다. 이를 행렬 형태로 정리하면 다음과 같다.

$$\frac{dV}{dt_b} = F_{Total}^{B} / m - w \times v \quad (14)$$

회전 운동은 각운동량 $h = Jw$에 대한 운동방정식으로부터 유도된다.

$$\frac{dh}{dt_b} = M \quad (15)$$

여기서 M은 외부 모멘트 벡터로, 모터의 추력 불균형 및 에어로 다이나믹 효과로 인해 발생한다. 바디좌표계로 변환하면 다음의 형태를 갖는다.

$$\frac{dh}{dt_b} + w \times h = M \quad (16)$$

$$h = \begin{bmatrix} J_x p \\ J_y q \\ J_z r \end{bmatrix}, \quad w = \begin{bmatrix} p \\ q \\ r \end{bmatrix} \quad (17)$$

따라서 각속도에 대한 미분방정식은 다음과 같이 표현된다.

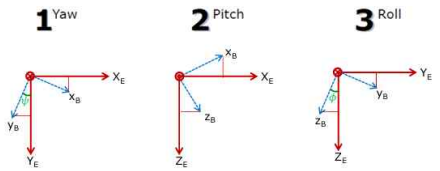
$$\dot{w} = J^{-1}(M - w \times Jw) \quad (18)$$

3-2 Kinematics

Kinematics 운동학 에서는 이너셜좌표계와 바디좌

표계 간의 좌표변환에 대해 기술한다.

이너셜좌표계는 x ,y_E, z_E)로 표시하고 바디좌표계는 (x_B, y_B, z_B)로 표시한다 여기서 z_E는 지구 기준 위쪽방향이며, z_B는 쿼드콥터기준 아랫방향이다. 두 좌표계간의 관계는 다음의 식들로 나타낼 수 있다.



$$R_z(\psi)=\begin{bmatrix} c(\psi) & -s(\psi) & 0 \\ s(\psi) & c(\psi) & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad R_y(\theta)=\begin{bmatrix} c(\theta) & 0 & s(\theta) \\ 0 & 1 & 0 \\ -s(\theta) & 0 & c(\theta) \end{bmatrix} \quad R_x(\varphi)=\begin{bmatrix} 1 & 0 & 0 \\ 0 & c(\varphi) & -s(\varphi) \\ 0 & s(\varphi) & c(\varphi) \end{bmatrix}$$

$$R_B^E = R_z \cdot R_y \cdot R_x = \begin{bmatrix} c(\psi)c(\theta) & c(\psi)s(\theta)s(\varphi)-s(\psi)c(\varphi) & c(\psi)s(\theta)c(\varphi)+s(\psi)s(\varphi) \\ s(\psi)c(\theta) & s(\psi)s(\theta)s(\varphi)+c(\psi)c(\varphi) & s(\psi)s(\theta)c(\varphi)-c(\psi)s(\varphi) \\ -s(\theta) & c(\theta)s(\varphi) & c(\theta)c(\varphi) \end{bmatrix}$$

Fig. 16. Transfer Function $\begin{matrix} E \\ B \end{matrix}$

R ,R_y, R_x는 각각 yaw,pitch,roll에 대한 회전행렬이며, 이를 다 곱하면 바디좌표계에서 이너셜좌표계로의 변환행렬 R_B^E가 만들어진다. 이의 역행렬 R_E^B이 이너셜좌표계에서 이너셜좌표계로의 변환행렬이다. 이는 R_B^E의 전치와 동일하다.

기체의 회전은 오일러 각을 사용하여 정의한다. 각속도에 대한 변환행렬은 다음과 같다.

$$\begin{bmatrix} p \\ q \\ r \end{bmatrix} = \begin{bmatrix} \dot{\phi} \\ 0 \\ 0 \end{bmatrix} + R_x^T(\phi)\begin{bmatrix} 0 \\ \dot{\theta} \\ 0 \end{bmatrix} + R_x^T(\phi)\cdot R_y^T(\theta)\begin{bmatrix} 0 \\ 0 \\ \dot{\psi} \end{bmatrix}$$

$$= \begin{pmatrix} \dot{\phi} \\ 0 \\ 0 \end{pmatrix} + \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & \sin\phi \\ 0 & -\sin\phi & \cos\phi \end{pmatrix}\begin{pmatrix} 0 \\ \dot{\theta} \\ 0 \end{pmatrix} + \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & \sin\phi \\ 0 & -\sin\phi & \cos\phi \end{pmatrix}\begin{pmatrix} \cos\theta & 0 & -\sin\theta \\ 0 & 1 & 0 \\ \sin\theta & 0 & \cos\theta \end{pmatrix}\begin{pmatrix} 0 \\ 0 \\ \dot{\psi} \end{pmatrix}$$

$$= \begin{pmatrix} 1 & 0 & -s\theta \\ 0 & c\phi & s\phi c\theta \\ 0 & -s\phi & c\phi c\theta \end{pmatrix}\begin{pmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{pmatrix}.$$

Inverting we get

$$\begin{pmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{pmatrix} = \begin{pmatrix} 1 & \sin(\phi)\tan(\theta) & \cos(\phi)\tan(\theta) \\ 0 & \cos(\phi) & -\sin(\phi) \\ 0 & \sin(\phi)\sec(\theta) & \cos(\phi)\sec(\theta) \end{pmatrix}\begin{pmatrix} p \\ q \\ r \end{pmatrix}.$$

Undefined @ $\theta = \pm\frac{\pi}{2}$

Fig. 17. Angular_velocity_transfer

3-3 시뮬링크 구성

시뮬링크구성은 전체적으로 Body Dynamic 부분과 Kinematic 부분으로 이루어져 있다. 이는 다음과 같다.

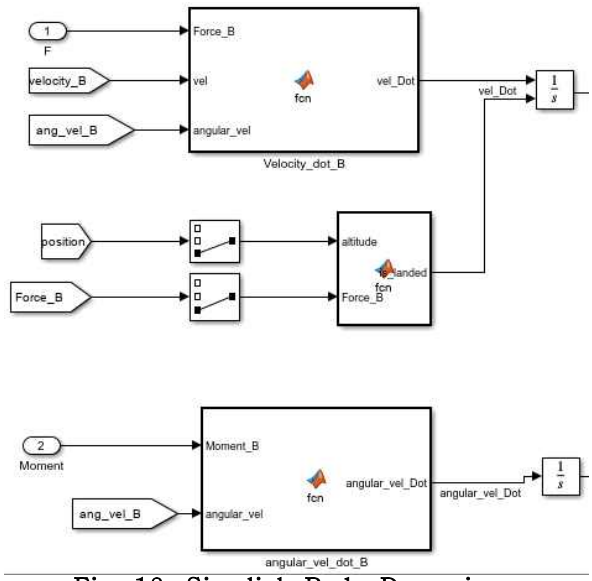


Fig. 18. Simulink Body Dynamics

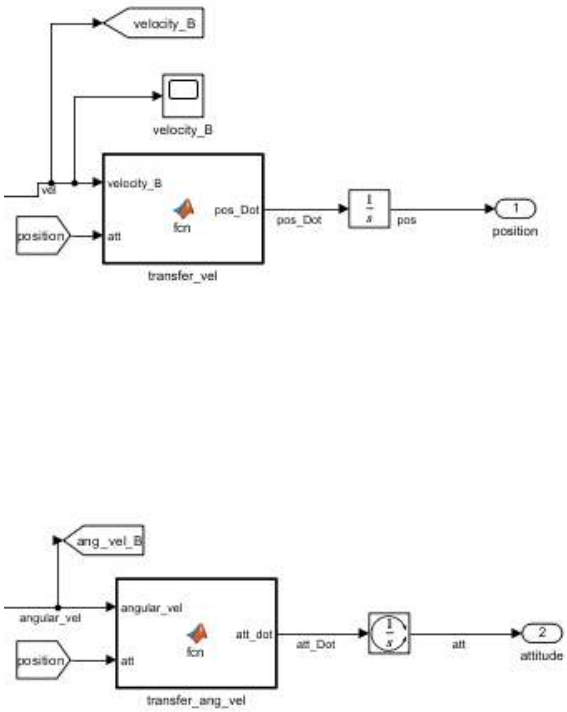


Fig. 19. Simulink Kinematics

기를 작성기를 작성

Body Dynamic시스템의 매트랩함수는 다음과 같다.

```
function vel_Dot = fcn(Force_B, vel, angular_vel)

m= 1.5;

vel_Dot =  Force_B / m - cross(angular_vel,vel);
```

Fig. 20. Velocity_dot_B

```
function angular_vel_Dot = fcn(Moment_B, angular_vel)

Inertia = diag([0.02 0.02 0.025]);

angular_vel_Dot = Inertia\ (Moment_B- cross(angular_vel,Inertia*angular_vel));
```

Fig. 21. angular_velocity_dot_B

velocity_dot_B 매트랩fcn 에서는 (14)식을 이용해 바디좌표계 각가속도를 도출하였으며 angular_velocity_dot_B fcn에서는 (18)식을 통해 바디좌표계 각 가속도를 도출 하였다. 이를 적분하여 바디좌표계 속도와 각속도를 구하였으며 이를 다시 매트랩fcn에 입력해주는 방식을 사용하였다.

Kinematic시스템의 매트랩fcn은 다음과 같이 구성했다.

```
function pos_Dot = fcn(velocity_B, att)

phi=att(1);
theta=att(2);
psi=att(3);

R = [cos(theta)*cos(psi)                          , cos(theta)*sin(psi)                          , -sin(theta);
     sin(phi)*sin(theta)*cos(psi)-cos(phi)*sin(psi), sin(phi)*sin(theta)*sin(psi)+cos(phi)*cos(psi), sin(phi)*cos(theta);
     cos(phi)*sin(theta)*cos(psi)+sin(phi)*sin(psi), cos(phi)*sin(theta)*sin(psi)-sin(phi)*cos(psi), cos(phi)*cos(theta)];

pos_Dot = R'*velocity_B;
```

Fig. 22. transfer_vel_fcn

```
function att_dot = fcn(angular_vel, att)

phi=att(1);
theta=att(2);

R = [1,     sin(phi)*tan(theta),   cos(phi)*tan(theta);
     0,                 cos(phi),           -sin(phi);
     0,     sin(phi)/cos(theta),   cos(phi)/cos(theta)];


att_dot = R * angular_vel;
```

Fig. 23. transger_ang_vel_fcn

fig16, fig17에서의 변환행렬을 사용해 kinematic시스템의 매트랩fcn을 작성하였고 이를 통해 이너셜좌표계의 속도와 각속도를 도출하였고 이를 적분하여 이너셜좌표계에서의 위치와 자세를 구하였다.

3-4 Landing detection

입력각속도가 들어오기 전까지 쿼드콥터는 중력의 영향을 받는다. 그러나 이때 시뮬레이션환경에서는 지면이 적용되어 있지 않기 때문에 위치가 하강하여 호버링 하는 결과를 볼 수 있다. 이를 해결하기 위해 body dynamics 시스템의 integrator 적분블록에 레벨유지 조건을 걸어 주었다. 이때 true,false를 입력하기 위해 landing detection fuction함수를 만들어주었다.

```
function is_landed = fcn(altitude, Force_B)

if altitude >= 0 && Force_B>0
    is_landed = true;
else
    is_landed=false;
end
```

Fig. 24. Landing detection fuction

먼저, From블록을 통해 위치와 Force_B 값을 입력해주었다. 이후 Selectior블록으로 position,Force_B벡터의 z요소를 뽑아내어 매틀랩fcn에 입력해주었다. 매트랩fcn에서는 고도가 0이상이고 추력이 0보다크면 true값을 반환 아니면 false값을 반환한다.

3-5 결과

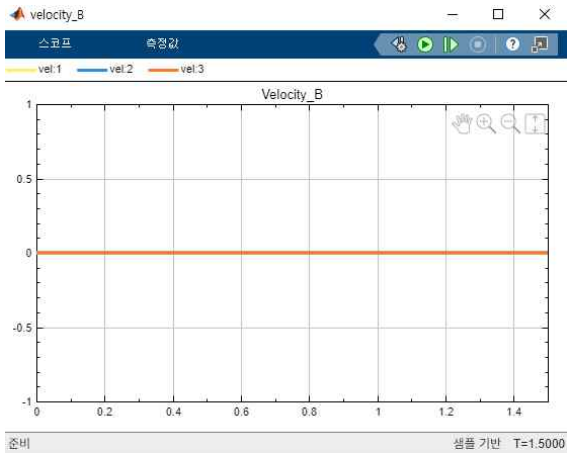


Fig. 24. Velocity_B_result

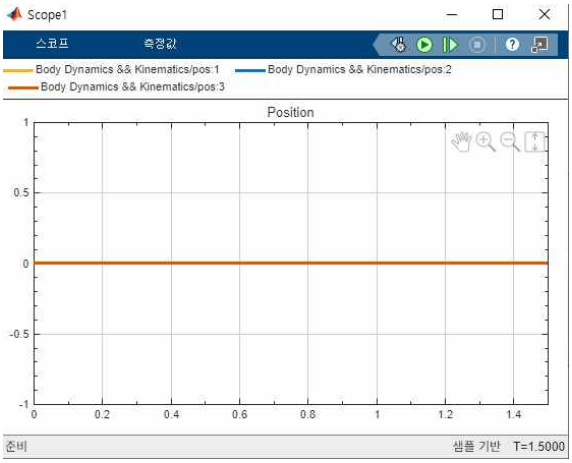


Fig. 25. Position_result

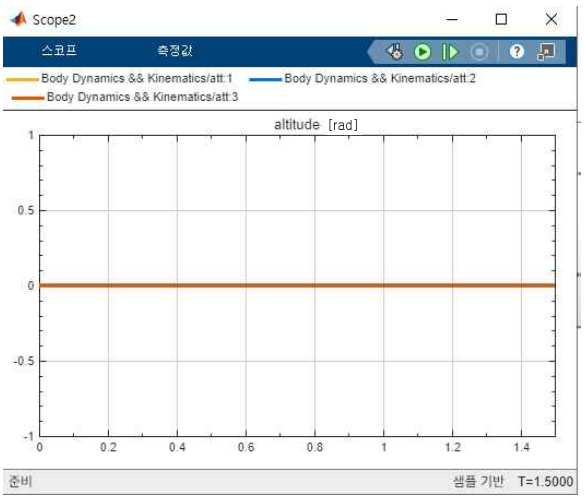


Fig. 26. attitude_result

호버링상태의 각속도를 4개모터 모두 동일하게 입력하였을 때 바디 좌표계 속도, 이너셜 좌표계 위치, 자세 모두 0으로 나옴을 확인할 수 있다. 이는 1초 이전에는 쿼드콥터가 지면에 있음으로 인식하여서 속도가 변하지 않으며, 1초 이후에는 호버링상태로 쿼드콥터의 Force,Moment 모두 0이기 때문이다.

4. 시뮬레이션2

시뮬레이션2에서는 시뮬레이션1의 호버링모터속도에 동일한 gain값 2를 추가하여 시뮬레이션하였다. 이때 4개의 모터속도는 모두 동일하다.

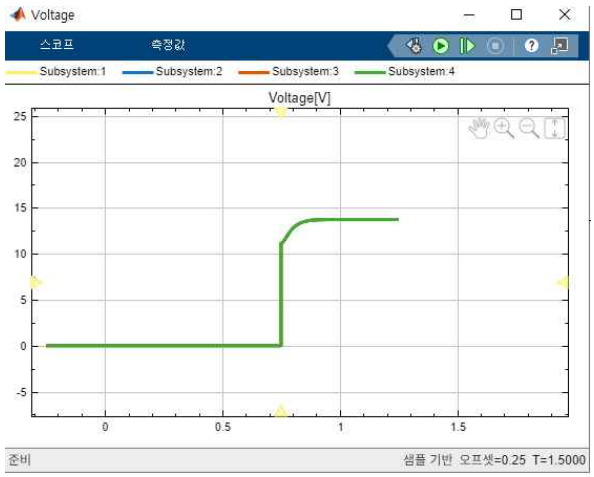


Fig. 27. Voltage_result

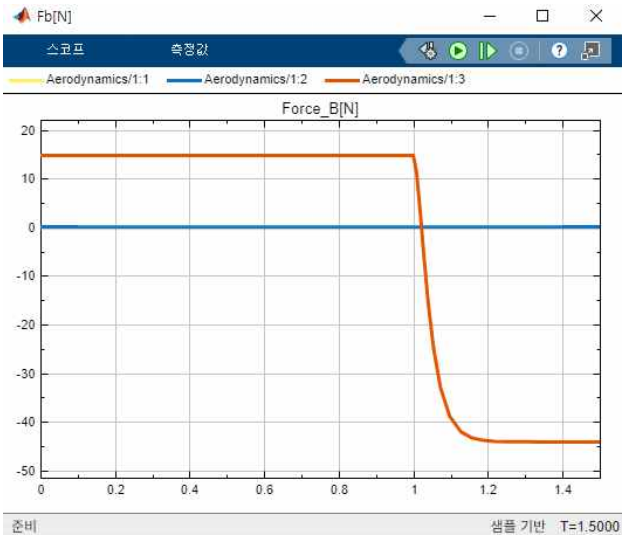


Fig. 28. Force_B_result

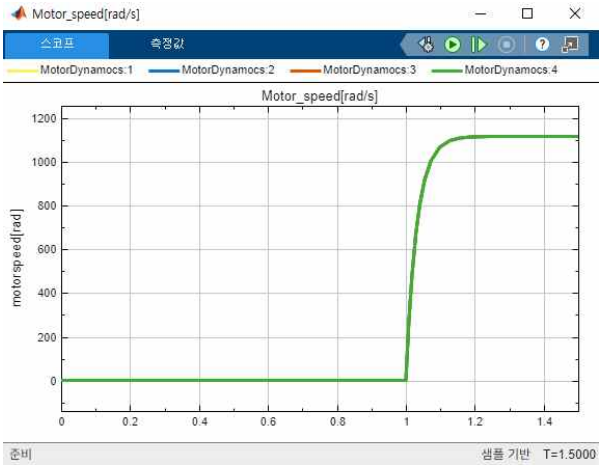


Fig. 29. Motor_speed_result

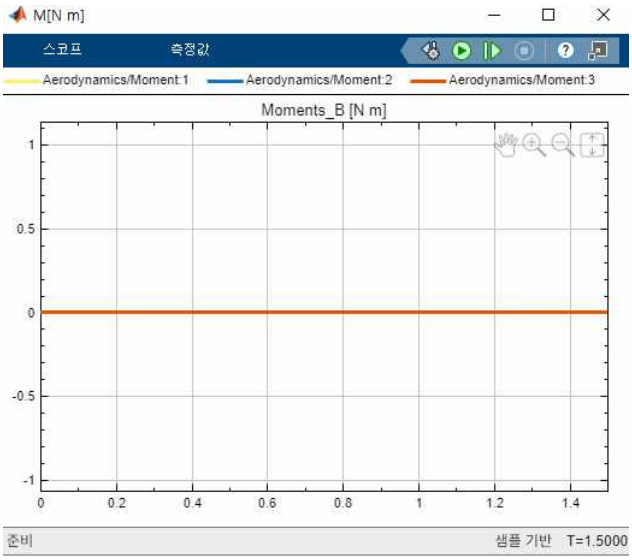


Fig. 30. Moments_B_result

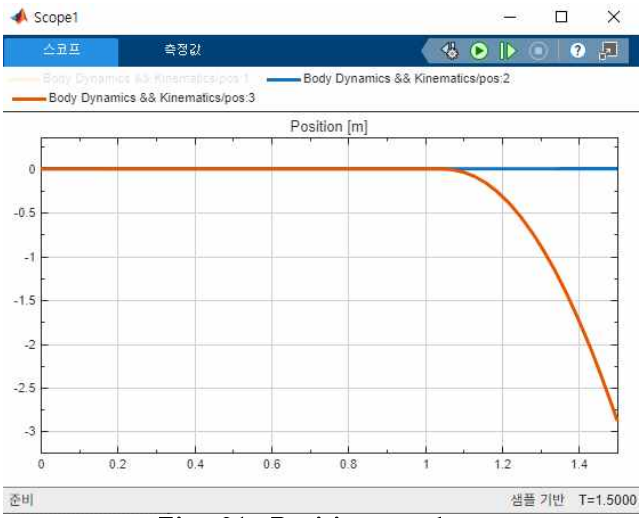


Fig. 31. Position_result

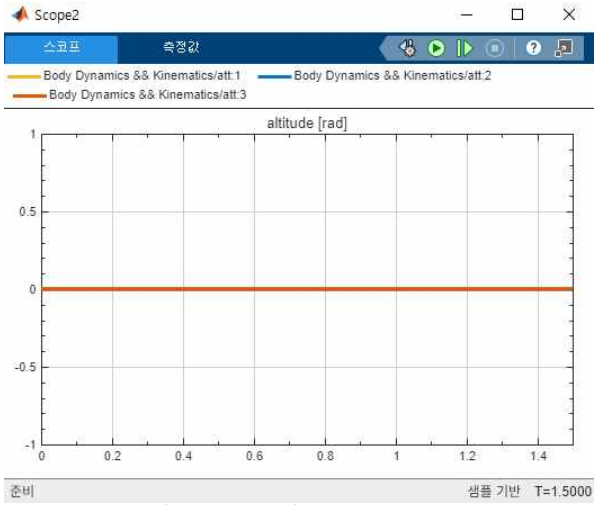


Fig. 32. attitude_result

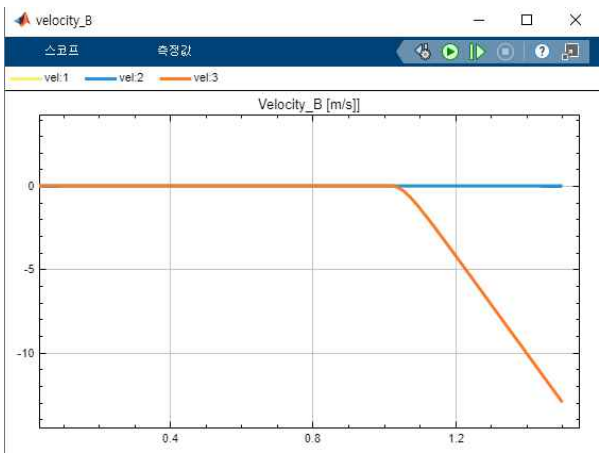


Fig. 33. Velocity_B_result

4개모터속도를 모두 2배하여 시뮬레이션한 결과 모터 각속도는 원래 호버링모터스피드의 두배인 1120rad/s으로 입력되었다. 이때 필요전압은 시뮬레이션1에서의 6의 두배정도로 계산 되었으며, 바디좌표계 힘과 모멘트는 각각 45[N],0[N m]으로 계산되었다 이는 식(6)에서 보여지는 것처럼 추력은 각속도 제곱에 비례하며, 모터의 회전방향이 대칭이므로 서로 각속도가 동일할 때 모멘트가 상쇄되기 때문이다. 따라서 z방향 속도는 선형적으로 감소하는 것을 확인할 수 있으며 , z축방향 위치는 기하급수적으로 감소함을 확인할 수 있다.

후 기

본 시뮬레이션 구축을 통해 쿼드콥터의 공기역학 이론을 학습하며, 프로펠러의 각속도 변화가 비행체에 작용하는 힘과 모멘트에 어떤 영향을 미치는지 보다 명확하게 이해할 수 있었다. 또한 Body Dynamics와 Kinematics을 공부하면서 서로 다른 좌표계 간의 상대 벡터 변환을 기반으로 한 동역학 이론을 다시 정립할 수 있었고, 이를

통해 학부 2학년 때 배웠던 동역학 개념을 실질적인 예제로 복습할 수 있는 좋은 기회가 되었다. 뿐만 아니라 모터 내부의 전기적 특성과 시스템을 심도 있게 학습함으로써 추진 시스템에 대한 이해가 더욱 확장되는 뜻깊은 경험이 되었다.

이번 연구 경험을 바탕으로 다양한 비행 환경을 시뮬레이션하고, 제어기 설계 및 검증에 대해 더 깊이 탐구해보고자 하는 의지가 생겼다. 또한 향후에는 모터뿐만 아니라 배터리 상태를 고려한 에너지 기반 제어 및 비행 지속성 향상 연구로까지 확장해 보고자 한다